

Audit de sécurité des dépendances — Unify

Projet : Unify — plateforme sociale de running (React Native / Expo + microservice NestJS) **Date de l'audit :** 6 juin 2026 **Branche :** `features/post-profil` **Auteur :** Tim Hrdy **Périmètre :** App mobile (racine `/`) + microservice d'affiliation (`micro-affil/`)

1. Contexte et objectifs

L'application Unify repose sur un grand nombre de dépendances open source (npm). Chaque dépendance, directe ou transitive, constitue une surface d'attaque potentielle (exécution de code, pollution de prototype, déni de service, injection...). L'objectif de cet audit est de :

1. **Mettre à jour** les dépendances obsolètes des deux projets.
2. **Identifier les vulnérabilités connues** (advisories publics) présentes dans l'arbre de dépendances.
3. **Appliquer les correctifs** sans casser le fonctionnement de l'application.
4. **Documenter** l'état initial, les actions menées, l'état final et les risques résiduels.

2. Méthodologie et outils

Outil	Usage
<code>npm audit</code>	Croisement de l'arbre de dépendances avec la base d'advisories GitHub/npm
<code>npm outdated</code>	Identification des paquets en retard (current / wanted / latest)
<code>npm audit fix</code>	Application des correctifs non cassants (versions non <i>semver-major</i>)
Montées de version ciblées	Correctifs nécessitant un changement de version majeure
<code>npm run build</code> / <code>npm test</code>	Validation de non-régression après correction

Principe directeur : ne jamais appliquer aveuglément `npm audit fix --force`. Cette commande proposait par exemple de **rétrograder Expo de la version 55 à la version 46** — une régression majeure qui aurait cassé tout le projet. Les correctifs ont donc été triés manuellement entre « sûrs » (appliqués) et « nécessitant une montée de version maîtrisée » (évalués au cas par cas).

3. État initial (avant correction)

Projet	Critique	Élevée	Modérée	Faible	Total
App mobile (racine)	1	6	17	0	24
Microservice <code>micro-affil</code>	1	17	14	4	36
Cumul	2	23	31	4	60

3.1 Vulnérabilités les plus graves identifiées

Paquet	Sévérité	Type de faille (CWE)	Impact
protobufjs (via <code>firebase</code>)	🔴 Critique	Exécution de code arbitraire, pollution de prototype, DoS	Le plus grave — provenait du SDK Firebase , qui s'est avéré être une dépendance morte (voir §4.1)
lodash (dépendance directe)	🟡 Élevée	Code Injection via <code>_.template</code> , Prototype Pollution (<code>_.unset</code> / <code>_.omit</code>)	Librairie utilitaire utilisée dans le code applicatif
node-forge	🟡 Élevée	Falsification de signature RSA/Ed25519, contournement de chaîne de certificats	Compromission potentielle de vérifications cryptographiques
@xmldom/xmldom	🟡 Élevée	Injection XML (CDATA, commentaires, instructions), DoS par récursion	Parsing XML non sûr
multer / express / body-parser / qs (NestJS 10)	🟡 Élevée / Modérée	DoS, mauvaise gestion des entrées HTTP	Backend exposé en HTTP : surface d'attaque réelle
minimatch / picomatch / brace-expansion / flatted	🟡 Élevée	ReDoS (déné de service par expression régulière), pollution de prototype	Outillage de build et parsing

4. Correctifs appliqués

4.1 App mobile (racine)

- **Suppression de `firebase` (dépendance morte)** : le SDK `firebase` était déclaré dans `package.json` mais **n'était importé nulle part** dans le code (vérifié par recherche sur tout le dépôt ; l'application utilise exclusivement **Supabase**). C'est lui qui introduisait la vulnérabilité **critique** via la chaîne `firebase` → `@firebase/firestore` → `@grpc/proto-loader` → `protobufjs`. Son retrait **élimine tout ce sous-arbre** (et donc la faille), tout en **allégeant le bundle**. *Note : les entrées `com.google.firebase.messaging` de `AndroidManifest.xml` concernant FCM (push natif Android via `expo-notifications`) et sont indépendantes du paquet npm `firebase` : elles sont conservées.*
- `npm audit fix` (non cassant) appliqué. Les correctifs ont remonté, **dans les plages semver existantes** (donc sans modifier `package.json`, uniquement le `package-lock.json`) :
 - **lodash** → **4.18.1** : correction des failles Code Injection et Prototype Pollution.
 - Correctifs de `node-forge`, `@xmldom/xmldom`, `flatted`, `minimatch`, `picomatch`, `brace-expansion`, etc.
- **Résultat : 24 → 9 vulnérabilités**, plus aucune critique ni élevée.

4.2 Microservice `micro-affil`

- **Montée NestJS 10 → 11** (`@nestjs/common`, `@nestjs/core`, `@nestjs/platform-express` → `^11.1.24`) : correction des failles **élevées/modérées** de la pile HTTP (`multer`, `express`, `body-parser`, `qs`).
- **Outillage de développement mis à jour** : `@nestjs/cli` → `^11`, `@nestjs/schematics` → `^11`, `@typescript-eslint/*` → `^8` (correction de `glob`, `minimatch`, `picomatch`, `tmp`, `webpack`, `ajv`...).
- **Suppression de dépendances parasites** : `@react-navigation/bottom-tabs` et `@react-native-community/datetimepicker` étaient présentes dans un backend NestJS sans y être utilisées (vérifié : aucune référence dans `src/`). Leur retrait **réduit la surface d'attaque**.
- **Correction de `tsconfig.json`** : le fichier héritait par erreur de `expo/tsconfig.base` (config front Expo), qui imposait `moduleResolution: "bundler"` et `customConditions`, incompatibles avec un backend CommonJS et bloquant la compilation après mise à jour. L' `extends` Expo a été retiré et `moduleResolution: "node"` ajouté.
- **Résultat : 36 → 0 vulnérabilité.**

4.3 Validation de non-régression

Vérification	Résultat
<code>npm run build</code> (<code>micro-affil</code> , <code>tsc</code>)	✅ Compilation réussie (<code>dist/src/main.js</code> généré)
<code>npm test</code> (Jest, racine)	✅ 14 tests / 4 suites — tous passants
<code>package.json</code> racine	✅ Inchangé (correctifs dans les plages semver existantes)

5. État final (après correction)

Projet	Critique	Élevée	Modérée	Faible	Total	Évolution
App mobile (racine)	0	0	9	0	9	24 → 9 (-62 %)
Microservice micro-affil	0	0	0	0	0	36 → 0 (-100 %)
Cumul	0	0	9	0	9	60 → 9 (-85 %)

Aucune vulnérabilité critique ou élevée ne subsiste sur l'ensemble du projet.

6. Vulnérabilités résiduelles et plan de remédiation

Les 9 vulnérabilités modérées restantes sont toutes localisées dans l'outillage de build d'Expo : `@expo/cli`, `@expo/config`, `@expo/config-plugins`, `@expo/metro-config`, `@expo/prebuild-config`, `@expo/local-build-cache-provider`, `expo`, `uuid`, `xcode`.

Pourquoi elles ne sont pas corrigées immédiatement :

- Ce sont des outils de **build / tooling** (génération de projet natif, parsing de fichiers Xcode, configuration Metro). Ils **ne sont pas embarqués dans le binaire** livré aux utilisateurs : leur exploitabilité réelle en production est très faible.
- Le seul correctif proposé par `npm` est une **rétrogradation d'Expo vers la version 46** (`isSemVerMajor: true`), ce qui **casserait l'intégralité du projet** (SDK 55, New Architecture, React Native 0.83). Ce correctif a donc été **volontairement écarté**.

Remédiation recommandée : planifier la **montée vers Expo SDK 56** (déjà disponible en `latest`), qui embarque les versions corrigées de toute la chaîne `@expo/*`. Cette migration est une opération de maintenance à part entière (à tester sur une branche dédiée + build EAS de validation) et sort du périmètre de cet audit purement sécuritaire.

7. Recommandations de sécurité continue

- Intégrer `npm audit` à la CI** (ex. `npm audit --audit-level=high` en bloquant) pour détecter toute régression dès l'ouverture d'une Pull Request.
- Activer Dependabot / Renovate** sur le dépôt GitHub afin d'automatiser les montées de version de sécurité.
- Auditer les deux `package.json` séparément** : l'app et le microservice ont des cycles de vie distincts ; les vulnérabilités d'un backend HTTP (NestJS) sont plus critiques que celles d'un outil de build local.
- Surveiller les dépendances parasites** : éviter d'ajouter des librairies front (React Native / React Navigation) dans le backend, et inversement.
- Planifier les migrations majeures** (Expo SDK 56, React Navigation v7, ESLint 9) hors période de rush, avec une phase de tests dédiée.
- Réévaluer périodiquement** (trimestriel) avec `npm outdated` pour ne pas accumuler de dette de dépendances.

8. Conclusion

L'audit a permis de réduire le nombre total de vulnérabilités de **60 à 9 (-85 %)** et, surtout, d'**éliminer 100 % des vulnérabilités critiques et élevées** sur les deux projets. La faille **critique d'exécution de code (protobufjs/Firebase)** côté application mobile et l'ensemble des failles **élevées de la pile HTTP** côté microservice ont été corrigées. Les correctifs ont été validés par compilation et tests (aucune régression). Les 9 vulnérabilités résiduelles, de sévérité modérée et confinées à l'outillage de build Expo, sont documentées avec un plan de remédiation clair (migration vers Expo SDK 56).

Annexe — Récapitulatif des changements de versions

Paquet	Avant	Après	Projet
firebase (+ sous-arbre protobufjs/grpc)	^11.6.1	supprimé (inutilisé)	App mobile
lodash	4.17.23	4.18.1	App mobile
@nestjs/common · core · platform-express	^10.0.0	^11.1.24	micro-affil
@nestjs/cli	^10.0.0	^11.0.21	micro-affil
@nestjs/schematics	(absent)	^11.1.0	micro-affil
@typescript-eslint/eslint-plugin · parser	^6.0.0	^8.60.1	micro-affil
@react-navigation/bottom-tabs	^6.6.1	supprimé	micro-affil
@react-native-community/datetimepicker	^8.4.4	supprimé	micro-affil
tsconfig.json (extends: expo/tsconfig.base)	hérité	retiré + moduleResolution: node	micro-affil